

```
import numpy as np
import pandas as pd
from scipy.stats import gaussian_kde
import scipy.stats as stats

import matplotlib.pyplot as plt
from matplotlib.ticker import StrMethodFormatter
```

Table of contents

1. [Descriptive and inferential statistics](#)
2. [Variables and data types](#)
3. [Probability – the chance of everything happening](#)
4. [Probability mass and probability density functions](#)
5. [Distributions in statistics](#)
 - [Normal distribution](#)
 - [Binomial distribution](#)
 - [Poisson distribution](#)
 - [Negative binomial distribution](#)
 - [Exponential distribution](#)
 - [Uniform distribution](#)

Introduction

Within the realm of data, probabilities and distributions are extremely important. These concepts will provide a vital basis when moving on to related technical fields such as machine learning, modelling, and algorithmic development.

Descriptive and inferential statistics

The two main branches of statistics are **descriptive** and **inferential** statistics.

The best comparison between the two is described in Keone Hon's *Introduction to Statistics*: "Descriptive statistics is used to say something about a set of information that has been collected only. Inferential statistics is used to make predictions or comparisons about a larger group (a population) using information gathered about a small part of that population. Thus, inferential statistics involves generalising beyond the data, something that descriptive statistics does not do."

Descriptive statistics

Descriptive statistics summarise and describe the main features of a dataset. This includes **measures of central tendency** (mean, median, mode) and **measures of variability** (range, variance, standard deviation).

Let's examine a simple dataset on deforestation rates in various regions to calculate basic descriptive statistics for the year 2020.

```
# Deforestation rates (in hectares) in various regions
data = {
    'Region': ['Amazon', 'Congo', 'Southeast Asia', 'Boreal Forests'],
    'Deforestation_rate_2020': [1000, 500, 750, 200], # Ratio
    'Deforestation_rate_2021': [800, 450, 700, 150], # Ratio
    'Region_type': ['Tropical', 'Tropical', 'Tropical', 'Temperate']
}
df = pd.DataFrame(data)

# Calculate basic descriptive statistics
descriptive_stats = df['Deforestation_rate_2020'].describe()
print(descriptive_stats)
```

count	4.00000
mean	612.50000
std	342.47871
min	200.00000
25%	425.00000
50%	625.00000
75%	812.50000
max	1000.00000

Name: Deforestation_rate_2020, dtype: float64

Inferential statistics

Inferential statistics involve making predictions or inferences about a population based on a sample of data. It includes hypothesis testing, confidence intervals, and regression analysis.

Building on our descriptive statistics, let's test the hypothesis that deforestation rates have decreased in 2021 due to new environmental policies by comparing 2020 and 2021 data.

```
# Calculate mean deforestation rates
mean_deforestation_2020 = df["Deforestation_rate_2020"].mean()
mean_deforestation_2021 = df["Deforestation_rate_2021"].mean()

years = ["2020", "2021"]
means = [mean_deforestation_2020, mean_deforestation_2021]

# Create figure
fig, ax = plt.subplots(figsize=(8, 4), dpi=100)

bars = ax.bar(
    years,
    means,
```

```

        width=0.2,
        edgecolor="white",
        linewidth=1.5
    )

    # Value labels
    for bar in bars:
        height = bar.get_height()
        ax.text(
            bar.get_x() + bar.get_width()/2,
            height,
            f"{height:.2f}",
            ha="center",
            va="bottom",
            fontsize=10,
            fontweight="bold"
        )

    # Formatting
    ax.set_title(
        "Average Deforestation Rate\n2020 vs 2021",
        fontsize=14,
        fontweight="bold",
        pad=12
    )

    ax.set_xlabel("Year", fontsize=12)
    ax.set_ylabel("Mean Deforestation Rate (hectares)", fontsize=12)

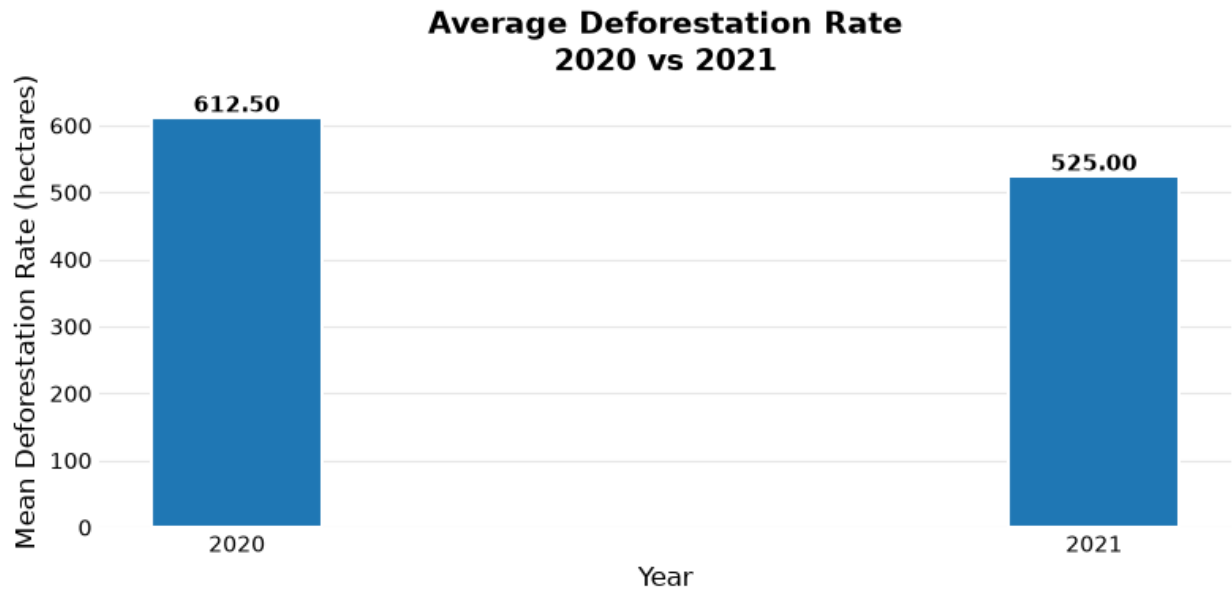
    ax.yaxis.set_major_formatter(StrMethodFormatter("{x:,.0f}"))

    ax.grid(axis="y", alpha=0.3)
    ax.set_axisbelow(True)

    # Clean style
    for spine in ax.spines.values():
        spine.set_visible(False)
    ax.tick_params(axis = 'both', length=0)

    plt.tight_layout()
    plt.show()

```



The result suggests that there might have been an improvement in the deforestation situation from 2020 to 2021, potentially due to new environmental policies or other factors.

However, while the bar chart provides a straightforward visual comparison, it does not indicate whether the difference in means is statistically significant. To confirm if the observed difference is not due to random variation, a statistical test like the **t-test**, which is beyond the scope of this notebook, would be required

Variables and data types

In statistics, variables can be classified into two wide classes:

1. **Categorical**
2. **Numeric**

In the categorical class, we have:

1. **Nominal data**
 - Binary data – Two distinct outcomes (yes/no or true/false).
 - Categorical data – Many data types (primary/secondary/tertiary).
1. **Ordinal data** – A number used in scoring that implies an order. An example is a rank from low to high.

In the numeric class, we have:

1. **Continuous data**
 - Interval (numerical) data – Lacks an absolute zero point (temperature in Celsius, year).
 - Ratio – Variable with any number (rainfall, time, temperature in Kelvin).
1. **Discrete data** – Variables which hold a whole number and can be used as a counting number (number of species in a forest).

```

df['Average_Rainfall_mm'] = [2300, 1600, 2500, 500]
df['Protected_Area'] = [True, False, True, True]
df['Biodiversity_Index'] = [8.5, 7.0, 9.0, 6.5]
df['Conservation_Status'] = ['High', 'Medium', 'Low', 'Medium']
# Mapping an ordinal variable to an ordered numerical representation
conservation_mapping = {'Low': 1, 'Medium': 2, 'High': 3}
df['Conservation_Status_Mapped'] =
df['Conservation_Status'].map(conservation_mapping)

df

```

	Region	Deforestation_rate_2020	Deforestation_rate_2021 \
0	Amazon	1000	800
1	Congo	500	450
2	Southeast Asia	750	700
3	Boreal Forests	200	150

	Region_type	Average_Rainfall_mm	Protected_Area	Biodiversity_Index \
0	Tropical	2300	True	8.5
1	Tropical	1600	False	7.0
2	Tropical	2500	True	9.0
3	Temperate	500	True	6.5

	Conservation_Status	Conservation_Status_Mapped
0	High	3
1	Medium	2
2	Low	1
3	Medium	2

We've introduced an ordinal variable, `Conservation_Status`, to rank regions based on their conservation priority. This variable has an inherent order: Low < Medium < High. We then map these ordinal values to numbers to facilitate statistical analysis, demonstrating a common technique for handling ordinal data in Python.

Understanding these variable types and their implications is crucial for accurately analysing and interpreting data. For instance, when analysing deforestation rates, knowing that these rates are ratio variables allows us to calculate meaningful ratios and averages, and perform other statistical analyses that depend on a true zero point. Similarly, recognising `Region_type` as a nominal variable informs us that statistical tests for independence might be appropriate for analysing relationships involving this variable.

Probability – the chance of everything happening

This section is going to introduce the world of probability and illustrate how powerful Python packages can help to compute the numbers related to probability.

It's useful to take a look at basic probabilities in *Chapter 3* of [Keone's textbook](#). We can also see how those concepts are implemented in Python [here](#).

Random variables

In probability theory, we define the concept of the sample space S as the set of all possible events; we can think of this as representing all of the possible outcomes of an experiment of which the outcome is unknown at the outset. For example, when getting dressed in the morning, we reach blindly into the drawer to pull out a pair of socks, knowing that there are white, black, and blue socks in there.

We can think of this as an experiment: We know what all the possible outcomes are, and if we know something about how many pairs of each colour sock we own, we also know something about the probabilities of those outcomes. But we don't know the actual outcome until we pull our hand back out and examine the pair of socks it's clutching.

We can define a variable X to stand for the values of the possible outcomes. This X is what we define as a **random variable**: one which can take on a range of possible values and its actual value is unknown until completion of the experiment.

Other classic examples include the tossing of a coin or the rolling of a die.

Discrete random variables

Discrete random variables can only take on specific values. Our sock, coin, and die examples above are all discrete: the possibilities are {white, black, blue}, {heads, tails}, and {1, 2, 3, 4, 5, 6} respectively.

Usually, we'll be looking at problems where the outcomes are real numbers, as in the die example. Where they're not, as in the coin example, we would usually proceed by assigning a numeric value to each of the possible outcomes, for example, 1 for heads and 2 for tails. Most common discrete random variables take on integer values, but that's not necessarily the case. If either 2 or 3 people are coming over to share a pizza with me, and we share equally, then the possible outcomes for me are one-quarter and one-third of a pizza.

Probability mass and probability density functions

We need some mathematical language to describe the probabilities of the various possible outcomes of random variables (what we refer to as **probability distributions**). Random variables have functions that map their outcomes onto their corresponding probabilities. These are most easily described in the case of discrete random variables, whose **probability mass functions** $p(x)$ directly map a given outcome x onto the probability of x being observed in a given trial.

There are three conditions that a probability mass function (PMF) must fulfil:

1. $p(x)$ is defined for all x , but $p(x) > 0$ only for a finite set of points. (Or a countably infinite set of points, but that's a subtlety we can happily ignore at this stage.)
2. $0 \leq p(x) \leq 1$ for all x . Intuitively, the bounds on a probability are 0 for an outcome that can never be observed and 1 for a certain outcome.

3. $\sum_x p(x)=1$. Observing one of the range of all possible outcomes is certain, and hence probabilities must sum to 1.

Consider the coin toss, coding heads as 1 and tails as 2. We have that $p(1)=p(2)=0.5$ and $p(x)=0$ for all $x \neq 1, 2$, hence all three conditions hold and the probability mass function for the coin toss is a valid PMF.

Continuous random variables are a little trickier, conceptually. For discrete variables, the probability mass function tells us directly the probability of individual outcomes, or (by summing) of groups of outcomes.

For example, the PMF for the roll of a die is $p(x)=1/6$ for $x \in 1, 2, 3, 4, 5, 6$.

We can see directly that there is a one-in-six chance of rolling a three, or a 50% chance ($1/6+1/6+1/6$) of rolling a four or above. But for continuous random variables, the probability of achieving a very specific outcome is in fact zero. This sounds a little counter intuitive, I know, but it has to do with the infinite divisibility of outcomes. Therefore, for a continuous random variable, we must think in terms of the probability of achieving an outcome between two limits. For example, for the 400 m world record, we could think of the probability of Wayde van Niekerk running another 43.03-second race, rounded to the nearest hundredth of a second, as the probability of time between 43.025 and 43.035 seconds.

We, therefore, can't directly define a probability mass function for continuous random variables; instead, we define what is known as a **probability density function** (PDF). There are also three conditions for PDFs which are conventionally written as $f(x)$:

1. $f(x)$ must be defined for all values of x .
2. All values of $f(x)$ are non-negative (note there is no upper bound on this, unlike $p(x)$ for a discrete random variable).
3. Finally, $\int_{-\infty}^{\infty} f(x) dx = 1$.

The last condition is the equivalent of the requirement that all probability masses sum to 1 for a discrete random variable. Probabilities for continuous variables are found by integration, which is after all the continuous analogue to summation, and this condition enforces that the integral of the probability densities over all possible values of x comes to 1, as necessary for a valid probability distribution. To continue with our 400 m example, we would evaluate the probability we're after by integrating between 43.025 and 43.035 seconds. We'd find the probability of Wayde breaking his own world record by integrating over the interval 0 (or negative infinity, but negative times are of course not possible so we can safely ignore this part of the real number line) to 43.025 seconds.

Cumulative distribution functions

Cumulative distribution functions (CDFs, or just distribution functions), usually denoted $F(x)$, give the probability that a random variable X takes on a value less than or equal to some given value x . In other words,

$$F(x) = P(X \leq x).$$

Before reading further, let's see if we can figure out how $F(x)$ would be defined for a discrete and a continuous random variable respectively.

That's right: For a discrete variable it would be

$$F(x) = \sum_{t \leq x} p(t),$$

and for a continuous random variable we would define the distribution function as

$$F(x) = \int_{-\infty}^x f(t) dt.$$

If CDFs are still not quite clear, go to [this link](#) for another explanation of CDFs.

Normal distribution

The Normal distribution, also known as the Gaussian distribution, is a cornerstone of statistical analysis and probability theory due to its **ubiquitous presence in natural and social phenomena**. Its significance lies in the **central limit theorem**, which states that the sum of many independent random variables, each with finite mean and variance, will tend to follow a Normal distribution, regardless of the original distribution of the variables. Here are the key features and the mathematical representation of the Normal distribution:

1. **Symmetry:** The normal distribution is perfectly symmetrical around its mean. This symmetry means that the mean, median, and mode of the distribution are equal.
2. **Bell-shaped curve:** It has a distinctive bell-shaped curve, where the probability density decreases rapidly and symmetrically as one moves away from the mean in either direction.
3. **Defined by two parameters:** The shape of the normal distribution is completely specified by two parameters – its mean (μ) and its standard deviation (σ). The mean determines the location of the centre of the graph, and the standard deviation determines the height and width of the graph.
4. **Asymptotic:** The tails of the normal distribution curve approach but never touch the horizontal axis. This indicates that no matter how far one moves away from the mean, the probability density never actually reaches zero.
5. **Inflection points:** The curve has inflection points at $(\mu - \sigma)$ and $(\mu + \sigma)$, where the curvature changes direction.

Mathematical representation

The probability density function (PDF) of the normal distribution for a random variable (X) is given by:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

where:

- (x) is the value of the random variable.
- (μ) is the mean of the distribution.
- (σ) is the standard deviation of the distribution.
- (e) is the base of the natural logarithm (≈ 2.71828).

Properties and usage

- **Standard normal distribution:** When $(\mu=0)$ and $(\sigma=1)$, the normal distribution is called the standard normal distribution. It is often denoted by (Z) and is used as a reference to transform other normal distributions using Z-scores. The PDF is shown below:

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$$

- **68-95-99.7 rule (empirical rule):** About 68% of the data falls within one standard deviation of the mean $(\mu \pm \sigma)$, 95% within two standard deviations $(\mu \pm 2\sigma)$, and 99.7% within three standard deviations $(\mu \pm 3\sigma)$.
- **Applicability:** The normal distribution is used in numerous statistical methods, including hypothesis testing, confidence intervals, and regression analysis. It's also fundamental in the natural and social sciences to describe real-world phenomena where the distribution of variables tends to be symmetrically distributed around a central value.

The normal distribution's mathematical elegance and practical applicability make it a pivotal concept in statistics, underlying many statistical methods and theories.

Notation-wise, if X is normally distributed with mean μ and standard deviation σ , then we denote this as $X \sim N(\mu, \sigma^2)$. Note that the second parameter is by convention shown as σ^2 rather than σ – this is the variance of the distribution.

Remember that for continuous random variables, to calculate the probability of an outcome in a particular range, we need to use the **cumulative distribution function**. As we've seen above, this requires integrating the **PDF**. While that is currently beyond the scope of this course, more information on integration is available here:

https://en.wikipedia.org/wiki/Numerical_integration. Fortunately, Python and `scipy` provide functions for us to evaluate PDFs without the need to integrate or even use the numerical tables that are sometimes provided.

Let's define a function to calculate the probability of observation following the above distribution lying between arbitrary bounds x_1 and x_2 , and then use that to calculate the probability of observation between 2 and 5. We use the simple logic here that the probability

we're after is the probability of observation of less than x_2 , less the probability of an observation less than x_1 .

Example 1

Now, suppose a study has shown that the optimal temperature range for the growth of a particular tree species involved in reforestation is between 18°C and 22°C. Temperatures outside this range can slow down growth or even cause stress to the trees.

Using the normal distribution, we can calculate not just the probability of exceeding a certain temperature but also the likelihood that the temperature will fall within the optimal range for tree growth.

```
# Add temperature data
df['Average_Temperature'] = [25, 24, 27, -5] # Average annual
temperature (°C)
df['Temperature_Std_Dev'] = [2, 1.5, 2.5, 3] # Standard deviation of
temperature
```

To analyse the impact of temperature variations on deforestation, we will model the temperature of each region using the normal distribution. This approach allows us to assess the likelihood of extreme temperature conditions that could exacerbate or mitigate deforestation rates.

For instance, we can calculate the probability of experiencing temperatures above a certain threshold that might lead to increased deforestation due to factors like increased fire risk or altered rainfall patterns.

The `norm` function from the `scipy.stats` module represents the normal (Gaussian) distribution. The `norm` function provides several methods for working with the normal distribution, including calculating probabilities, generating random variables, and more.

```
# Example: Probability of experiencing temperatures above 30°C in
Southeast Asia
region = 'Southeast Asia'
mean_temp = df.loc[df['Region'] == region,
'Average_Temperature'].values[0]
std_dev_temp = df.loc[df['Region'] == region,
'Temperature_Std_Dev'].values[0]

# Probability of temperature exceeding 30°C
prob_above_30 = 1 - stats.norm.cdf(30, mean_temp, std_dev_temp)
print(f"Probability of temperature above 30°C in {region}:
{prob_above_30:.2f}")
```

Probability of temperature above 30°C in Southeast Asia: 0.12

Example 2

Suppose we're interested in understanding the effectiveness of recent conservation initiatives. Based on our preliminary analysis, we estimate that the mean reduction in deforestation rates

since implementation is **250 hectares**, with a standard deviation of **50 hectares**. We are particularly interested in calculating the probability of observing a reduction greater than **300 hectares** in a region, as this would indicate a highly effective conservation effort.

To conduct this analysis, we will use the `norm.cdf` function from the `scipy.stats` package in Python, which will allow us to calculate the cumulative distribution function for a normal distribution. This approach will enable us to determine the likelihood of achieving a deforestation rate reduction of more than **300 hectares**, providing us with valuable insights into the effectiveness of conservation initiatives across the studied regions.

```
# Example: Probability of deforestation rate reduction
mean_reduction = 250
std_deviation = 50
# Assuming deforestation rate reduction follows a normal distribution
prob_greater_than_300 = 1 - stats.norm.cdf(300, mean_reduction,
std_deviation) # Mean reduction in hectares
# Probability of observing a reduction greater than 300 hectares
print(f'Probability of reduction greater than 300 hectares:
{prob_greater_than_300:.2f}') # Standard deviation

Probability of reduction greater than 300 hectares: 0.16
```

The `cdf` method of the `norm` function stands for **cumulative distribution function**. It gives the probability that a normally distributed random variable with the specified mean and standard deviation is less than or equal to a given value.

- `norm.cdf(300, mean_reduction, std_deviation)` calculates the probability that the reduction is less than or equal to 300 hectares.
- Subtracting this value from 1 gives the probability of the reduction being greater than 300 hectares because the total probability (the sum of the probability of an event and its complement) equals 1.
- If we were interested in seeing if the reduction is less than 300, the answer would be given by `norm.cdf(300, mean_reduction, std_deviation)`

Binomial distribution

The binomial distribution is a foundational concept in statistics, particularly useful in scenarios where we have a **series of independent experiments or trials** with two possible outcomes: **success** or **failure**. This distribution allows for the calculation of the probability of obtaining a specific number of successes in a fixed number of trials, assuming each trial is independent and the probability of success remains constant throughout the trials.

Key properties of the binomial distribution include;

1. **Discreteness:** The binomial distribution is discrete, meaning it calculates the probabilities of achieving a certain number of successes in a series of independent trials or events.
2. **Fixed number of trials:** The total number of trials (n) is fixed in advance. Each trial is independent of the others, and the outcome of one trial does not affect the outcomes of the other trials.

3. **Binary outcomes:** Each trial has only two possible outcomes, commonly referred to as "success" and "failure". The probability of success (p) is constant for each trial.
4. **Probability of success:** The probability of success on a single trial is denoted by p , while the probability of failure is denoted by q , where $q = 1 - p$.
5. **Sum of Bernoulli trials:** A binomial distribution can be thought of as the sum of n independent Bernoulli trials, each with the same probability of success p .

Mathematical representation

The probability of obtaining exactly k successes in n trials is given by the formula:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

where:

- $P(X = k)$ is the probability of getting exactly k successes.
- $\binom{n}{k}$ is the binomial coefficient, representing the number of ways to choose k successes from n trials.
- p is the probability of success on a single trial.
- $(1 - p)$ is the probability of failure on a single trial.

Consider a conservation project monitoring the presence of a certain endangered species in a set of habitat areas. Each area is checked for the presence of the species, with each check considered an independent trial.

- **n:** 50 habitat areas are surveyed.
- **p:** The probability of finding the species in any given area is 0.2 (20%).

Using the binomial distribution, we can calculate the probability of finding the species in exactly 10 of the 50 areas.

```
k = 10
n = 50 # Number of trials
p = 0.2 # Probability of success
prob_exactly_10 = stats.binom.pmf(k, n, p)
# Probability of exactly 10 successes (finding the species in 10 areas)
print(f'Probability of finding the species in exactly 10 out of 50 areas: {prob_exactly_10:.4f}')
```

```
Probability of finding the species in exactly 10 out of 50 areas:
0.1398
```

This calculation helps in understanding the likelihood of various outcomes, aiding in the assessment of conservation efforts' effectiveness and the planning of future actions based on probabilistic models.

```

n = 50
x = np.arange(0, n + 1)

fig, ax = plt.subplots(figsize=(8, 4), dpi=120)

for p in [0.2, 0.5, 0.8]:
    ax.plot(
        x,
        stats.binom.pmf(x, n, p),
        marker="o",
        linewidth=2,
        markersize=4,
        label=f"p = {p}"
    )

ax.set_title(
    "Binomial PMF for Different Probability Values",
    fontsize=14,
    fontweight="bold"
)

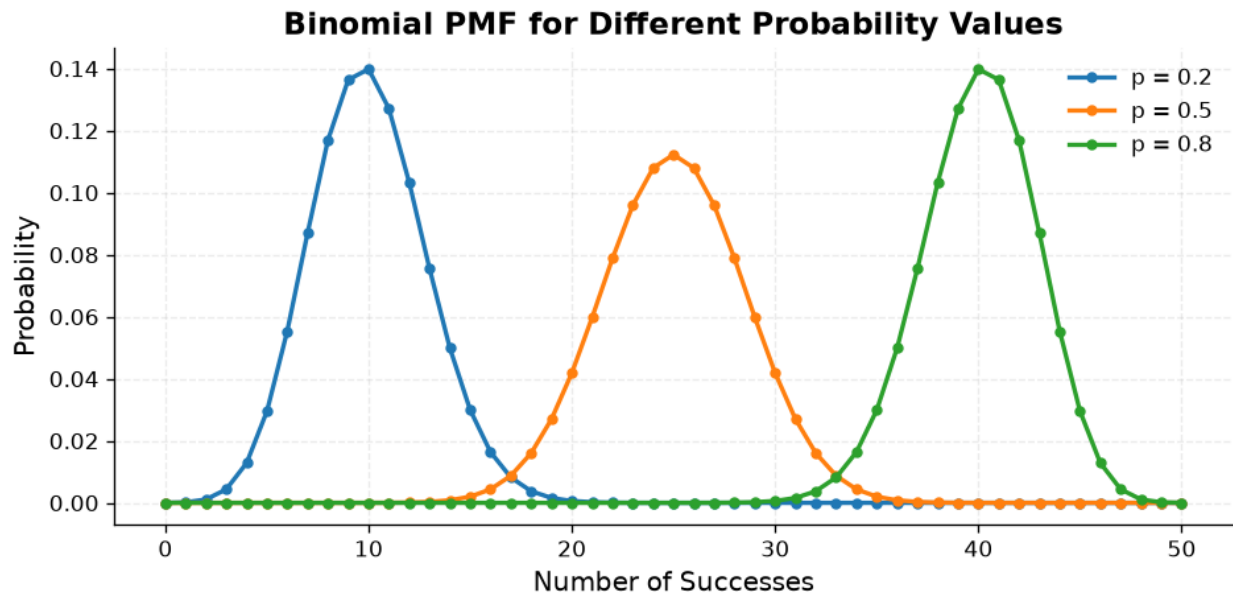
ax.set_xlabel("Number of Successes", fontsize=12)
ax.set_ylabel("Probability", fontsize=12)

ax.grid(alpha=0.3, color='0.75', ls="--")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.legend(frameon=False)

plt.tight_layout()
plt.show()

```



Poisson distribution

The Poisson distribution is particularly useful for modelling the number of events occurring within a fixed interval of time or space when these events happen **independently** of each other at a **constant average rate**. It's an ideal tool for analysing count data where the events are discrete and the number of occurrences in any given interval can range from zero to potentially infinity.

Key characteristics of the Poisson distribution include:

1. **Events occur independently:** The occurrence of one event does not affect the probability of another event occurring.
2. **Constant average rate:** Events occur at a constant mean rate (λ) over time or space.
3. **Discrete outcomes:** The number of events is counted in whole numbers.
4. **Unbounded upper limit:** There's no maximum limit to the number of events that can occur in the given interval (let $n \rightarrow \infty$).

Mathematical representation

The notation $X \sim P(\lambda)$ (that symbol $\sim$ is read "is distributed") signifies that X follows the Poisson distribution with parameter λ .

The probability of observing exactly (k) events in a fixed interval is given by the formula:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where:

- $P(X = k)$ is the probability of observing exactly (k) events.
- λ is the average rate (mean) of events per interval.
- e is the base of the natural logarithm (approximately 2.71828).

- $(k!)$ is the factorial of (k) .

Suppose an environmental agency has recorded an average rate (λ) of five illegal logging incidents per month in a particular region. We can use the Poisson distribution to calculate the probability of observing a specific number of incidents in a given month, which can inform enforcement and reforestation planning.

```
lambda_illegal_logging = 5 # Average number of illegal logging
incidents per month

# Probability of observing exactly 3 illegal logging incidents in a
month
prob_exactly_3 = stats.poisson.pmf(3, lambda_illegal_logging)
print(f"Probability of observing exactly 3 illegal logging incidents
in a month: {prob_exactly_3:.4f}")
```

Probability of observing exactly 3 illegal logging incidents in a month: 0.1404

```
# Parameters
poisson_lambda = 5
x = np.arange(0, 15)

# PMF
px = stats.poisson.pmf(x, poisson_lambda)

# Figure
fig, ax = plt.subplots(figsize=(8, 4), dpi=120)

# Stem plot
markerline, stemlines, baseline = ax.stem(
    x,
    px,
    linefmt="C0-",
    markerfmt="o",
    basefmt=" "
)

plt.setp(markerline, markersize=6)
plt.setp(stemlines, linewidth=1.6)

# Mean ( $\lambda$ )
ax.axvline(
    poisson_lambda,
    color="crimson",
    linestyle="--",
    linewidth=2,
    label=fr"$\lambda$ = {poisson_lambda}"
)

# Labels
```

```

ax.set_title(
    "Poisson Probability Mass Function",
    fontsize=14,
    fontweight="bold",
    pad=12
)

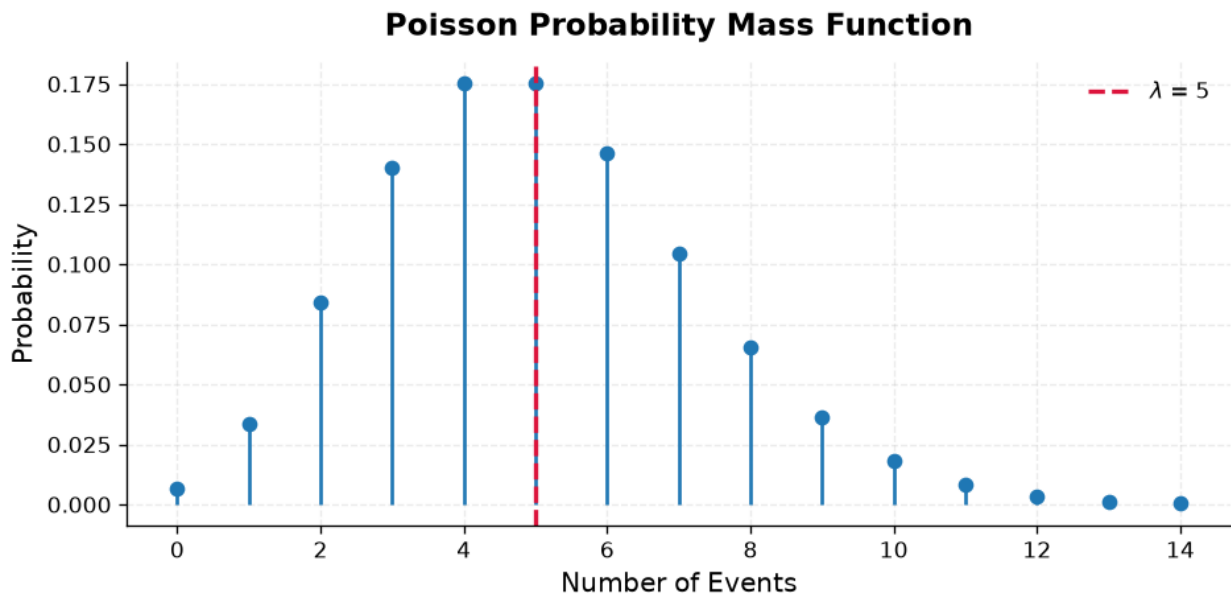
ax.set_xlabel("Number of Events", fontsize=12)
ax.set_ylabel("Probability", fontsize=12)

# Style
ax.grid(alpha=0.3, color='0.75', ls="--")
ax.set_axisbelow(True)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.legend(frameon=False)

plt.tight_layout()
plt.show()

```



Understanding the frequency of illegal logging incidents can help tailor reforestation efforts. For example, if the probability of a high number of incidents in certain months is significant, additional resources could be allocated for surveillance and reforestation during or following those periods.

```

# Probability of observing more than 7 illegal logging incidents in a
month
prob_more_than_7 = 1 - stats.poisson.cdf(7, lambda_illegal_logging)

```



```
print(f"Probability of observing more than 7 illegal logging incidents  
in a month: {prob_more_than_7:.4f}")
```

Probability of observing more than 7 illegal logging incidents in a
month: 0.1334

```
# Parameters
```

```
poisson_lambda = 7  
x = np.arange(0, 15)
```

```
# PMF
```

```
px = stats.poisson.pmf(x, poisson_lambda)
```

```
# Figure
```

```
fig, ax = plt.subplots(figsize=(8, 4), dpi=120)
```

```
# Stem plot
```

```
markerline, stemlines, baseline = ax.stem(  
    x,  
    px,  
    linefmt="C0-",  
    markerfmt="o",  
    basefmt=" "  
)
```

```
plt.setp(markerline, markersize=6)
```

```
plt.setp(stemlines, linewidth=1.6)
```

```
# Mean ( $\lambda$ )
```

```
ax.axvline(  
    poisson_lambda,  
    color="crimson",  
    linestyle="--",  
    linewidth=2,  
    label=fr"$\lambda$ = {poisson_lambda}"  
)
```

```
# Labels
```

```
ax.set_title(  
    "Poisson Probability Mass Function",  
    fontsize=14,  
    fontweight="bold",  
    pad=12  
)
```

```
ax.set_xlabel("Number of Events", fontsize=12)
```

```
ax.set_ylabel("Probability", fontsize=12)
```

```
# Style
```

```
ax.grid(alpha=0.3, color='0.75', ls="--")
```

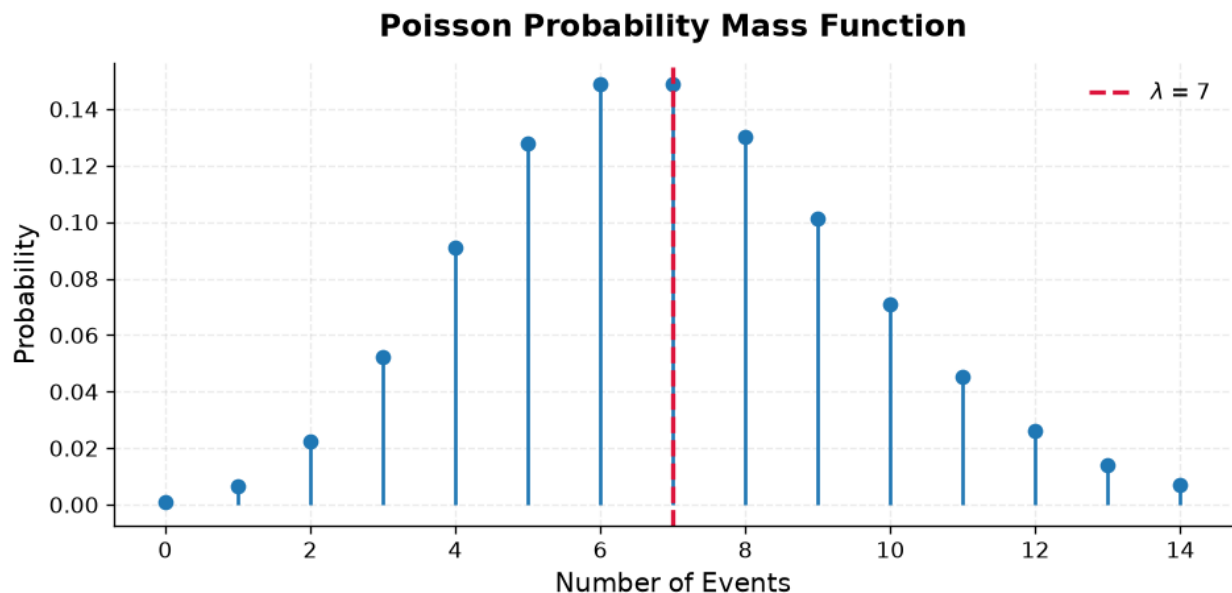
```

ax.set_axisbelow(True)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.legend(frameon=False)

plt.tight_layout()
plt.show()

```



This analysis assists in strategically planning reforestation projects, ensuring they are timed and resourced in a manner that accounts for the impact of illegal logging, thereby maximising the success rate of reforestation efforts

Negative binomial distribution

Suppose once again that we have a binomial process, where each trial is either a success or a failure. If we let X represent the number of **failures** that occur before we hit a specific number of **successes**, usually denoted r , and p is the **probability of success** on each trial, then $X \sim NB(r, p)$, i.e. X follows the negative binomial distribution with parameters r and p and has PMF:

$$\begin{aligned} p(x) &= \binom{x+r-1}{x} p^r (1-p)^x && \text{for } x = 0, 1, \dots, n \\ &= 0 && \text{otherwise} \end{aligned}$$

This PMF can be derived in an approach that is similar to the one used for the binomial distribution. Intuitively, each possible path that takes us to r successes and x failures must have probability $p^r (1-p)^x$. How many such combinations are there? Well, remember first of all that by our definition, the last trial must have been a success, so in the previous $x+r-1$ trials we will have had x failures.

There are $\binom{x+r-1}{x}$ ways of arranging these x failures and $r-1$ successes.

Let's plot the PMF for $p=0.3$ and r equal to either 2 or 3.

```
# Parameters
p = 0.3
x = np.arange(0, 11)

r1 = 2
r2 = 3

# PMFs
px1 = stats.nbinom.pmf(x, r1, p)
px2 = stats.nbinom.pmf(x, r2, p)

# Figure
fig, ax = plt.subplots(figsize=(8, 4), dpi=120)

# First distribution
markerline1, stemlines1, _ = ax.stem(
    x,
    px1,
    linefmt="C0-",
    markerfmt="o",
    basefmt=" "
)

# Second distribution
markerline2, stemlines2, _ = ax.stem(
    x,
    px2,
    linefmt="C1-",
    markerfmt="s",
    basefmt=" "
)

# Style stems
plt.setp(stemlines1, linewidth=1.8)
plt.setp(stemlines2, linewidth=1.8)
plt.setp(markerline1, markersize=6)
plt.setp(markerline2, markersize=6)

# Mean reference lines
mean_r1 = r1 * (1 - p) / p
mean_r2 = r2 * (1 - p) / p

ax.axvline(
    mean_r1,
    color="C0",
```

```

        linestyle="--",
        linewidth=2,
        alpha=0.8,
        label=f"Mean (r=2) = {mean_r1:.2f}"
    )

    ax.axvline(
        mean_r2,
        color="C1",
        linestyle="--",
        linewidth=2,
        alpha=0.8,
        label=f"Mean (r=3) = {mean_r2:.2f}"
    )

# Labels
ax.set_title(
    "Negative Binomial Probability Mass Function",
    fontsize=14,
    fontweight="bold",
    pad=12
)

ax.set_xlabel("Number of Failures", fontsize=12)
ax.set_ylabel("Probability", fontsize=12)

# Styling
ax.grid(axis="y", alpha=0.3)
ax.set_axisbelow(True)

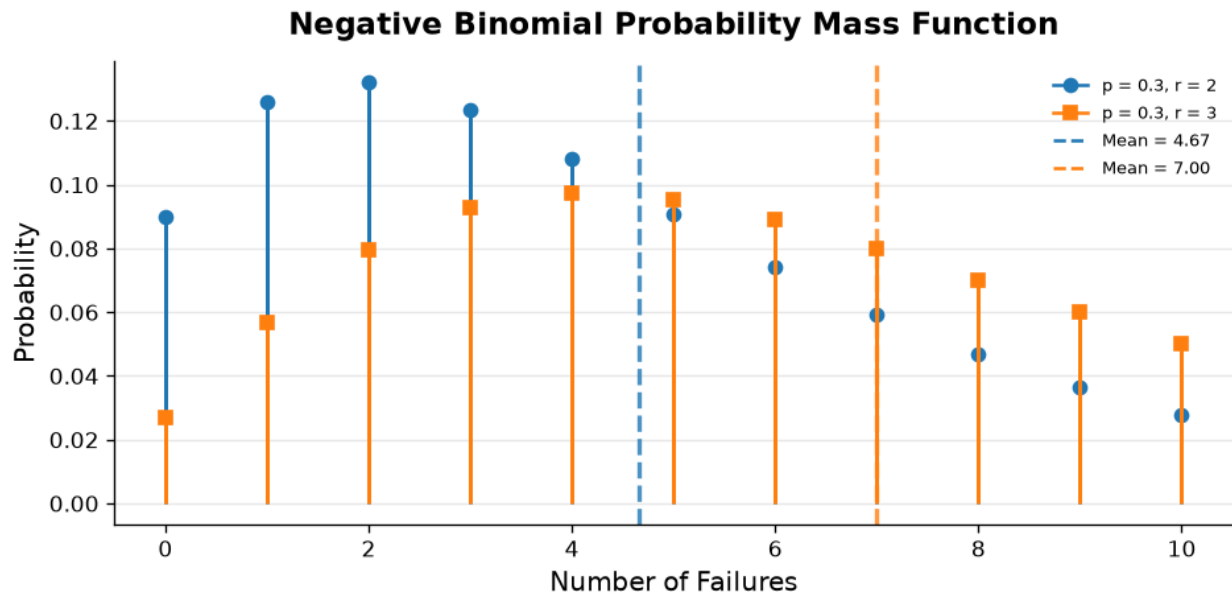
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

# Legend
handles = [
    plt.Line2D([], [], color="C0", marker="o", linestyle="-", label="p
= 0.3, r = 2"),
    plt.Line2D([], [], color="C1", marker="s", linestyle="-", label="p
= 0.3, r = 3"),
    plt.Line2D([], [], color="C0", linestyle="--", label=f"Mean =
{mean_r1:.2f}"),
    plt.Line2D([], [], color="C1", linestyle="--", label=f"Mean =
{mean_r2:.2f}")
]

ax.legend(handles=handles, frameon=False, fontsize=8)

plt.tight_layout()
plt.show()

```



Exponential distribution

The exponential distribution is related to the Poisson process which underlies the Poisson distribution (a Poisson process is simply one in which events occur at the rate of λ per time period).

Here, instead of modelling the number of events in a **time period**, as we did with the Poisson, we are interested in the **time interval** between events. Unlike the Poisson distribution, therefore, this is a **continuous random** variable.

Mathematical representation

With λ defined as before, then the PDF for $X \sim E(\lambda)$ measuring the time between events is:

$$\begin{aligned} f(x) &= \lambda e^{-\lambda x} && \text{for } x \geq 0 \\ &= 0 && \text{otherwise} \end{aligned}$$

for $(x \geq 0)$, where (λ) is the rate parameter and (e) is the base of the natural logarithm.

The exponential distribution is a crucial statistical tool for modelling the time between occurrences in continuous time processes, especially when these events happen independently and at a constant average rate.

Key features of the exponential distribution include:

1. **Memorylessness:** One of the unique properties of the exponential distribution is its *memoryless* nature. This means that the probability of an event occurring in the future is independent of how much time has already elapsed.
2. **Continuous time:** Unlike the Poisson distribution, which counts occurrences in fixed intervals, the exponential distribution deals with the time between these occurrences in a continuous space.

3. **Rate parameter (λ):** The rate at which events occur (average number of events in a time unit) is the inverse of the mean time between events.

Suppose a specific region is prone to forest fires, with an average occurrence **rate of 0.1** per year (or one fire every 10 years, on average). We can use the exponential distribution to calculate the **probability** of the next fire occurring within a certain timeframe, **say the next 5 years**, which is crucial for planning reforestation and preventive measures.

```
# Average time between forest fires is 10 years, so the rate  $\lambda = 0.1$ 
# per year
lambda_fire = 0.1
scale_fire = 1 / lambda_fire # Scale parameter for the exponential
# distribution is the inverse of  $\lambda$ 
```

```
# Probability of a forest fire occurring within the next 5 years
prob_within_5_years_fire = stats.expon.cdf(5, scale=scale_fire)
print(f"Probability of a forest fire occurring within the next 5
years: {prob_within_5_years_fire:.4f}")
```

Probability of a forest fire occurring within the next 5 years: 0.3935

```
# Parameters
```

```
lambda_var = 0.1
```

```
# X values
```

```
x = np.arange(0, 50, 0.01)
```

```
# Exponential CDF
```

```
cdf = stats.expon.cdf(x, scale=1 / lambda_var)
```

```
# Expected value
```

```
mean = 1 / lambda_var
```

```
# Plot
```

```
fig, ax = plt.subplots(figsize=(8, 4), dpi=120)
```

```
ax.plot(
    x,
    cdf,
    linewidth=2,
    label=r"$\lambda = 0.1$"
)
```

```
# Mark expected value
```

```
ax.axvline(
    mean,
    linestyle="--",
    linewidth=2,
    color = "crimson",
    label=f"Mean = {mean:.0f}"
)
```

```

# Formatting
ax.set_title(
    "Cumulative Distribution Function of an Exponential Distribution",
    fontsize=14,
    fontweight="bold",
    pad=12
)

ax.set_xlabel("Time / Waiting Interval", fontsize=12)
ax.set_ylabel("Cumulative Probability", fontsize=12)

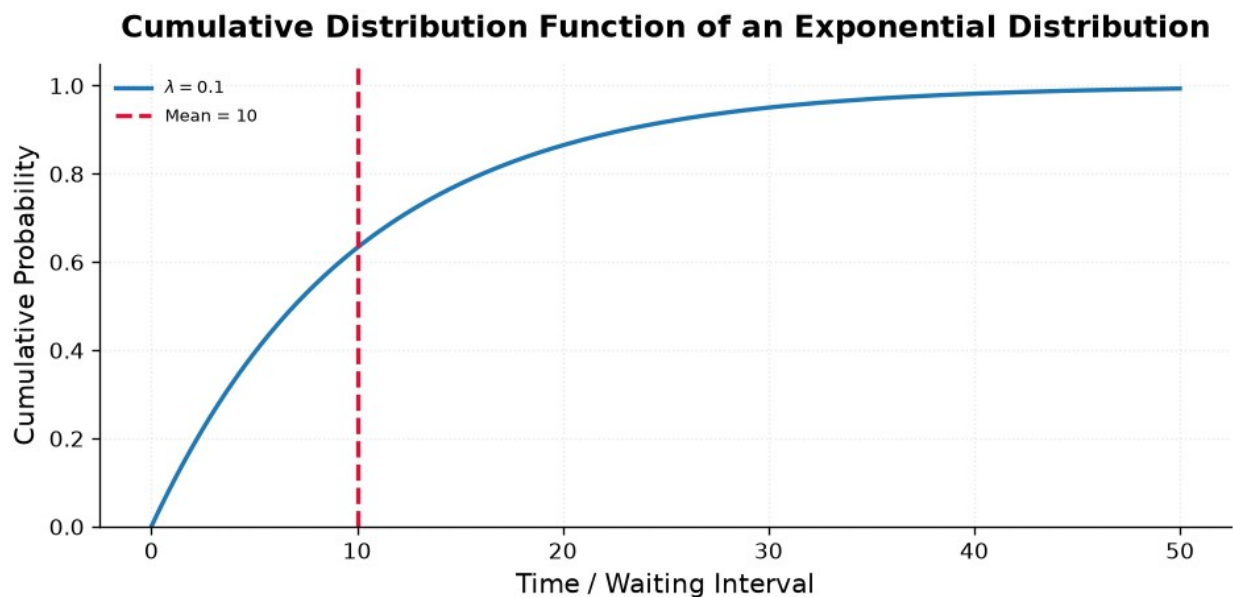
ax.set_ylim(0, 1.05)

# Style
ax.grid(alpha=0.3, ls=":", color="0.75")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.legend(frameon=False, fontsize=8)

plt.tight_layout()
plt.show()

```



Understanding the likelihood of forest fires (or other natural disasters) within a specific timeframe allows environmental planners to allocate resources more effectively. For example, if the probability of a fire occurring in the next five years is high, efforts can be focused on fire prevention measures, creating firebreaks, and selecting fire-resistant plant species for reforestation.

Uniform distribution

As the name suggests, a uniformly distributed random variable will take on any value in a given interval with equal probability. The two key parameters are therefore the lower and upper points of the interval, usually denoted a and b respectively, and $X \sim U(a, b)$ has PDF:

$$f(x) = \frac{1}{b-a} \text{ for } a \leq x \leq b$$

$$= 0 \text{ otherwise}$$

This looks like the following:

```
# Uniform distribution parameters
a = 0
b = 5

# Generate values
x = np.linspace(a - 1, b + 1, 500)

# PDF
y = stats.uniform.pdf(
    x,
    loc=a,
    scale=b-a
)

# Plot
fig, ax = plt.subplots(figsize=(8, 4), dpi=120)

# PDF curve
ax.plot(
    x,
    y,
    linewidth=2,
    label=f"Uniform({a}, {b})"
)

# Shade probability area
ax.fill_between(
    x,
    y,
    where=(x >= a) & (x <= b),
    alpha=0.2
)

# Mark boundaries
ax.axvline(
    a,
    linestyle="--",
    linewidth=2,
    color = 'crimson',
```



```

        label=f"a = {a}"
    )
    ax.axvline(
        b,
        linestyle="--",
        linewidth=2,
        color = 'crimson',
        label=f"b = {b}"
    )

# Labels
ax.set_title(
    "Probability Density Function of a Uniform Distribution",
    fontsize=14,
    fontweight="bold",
    pad=12
)

ax.set_xlabel(
    "Value",
    fontsize=12
)

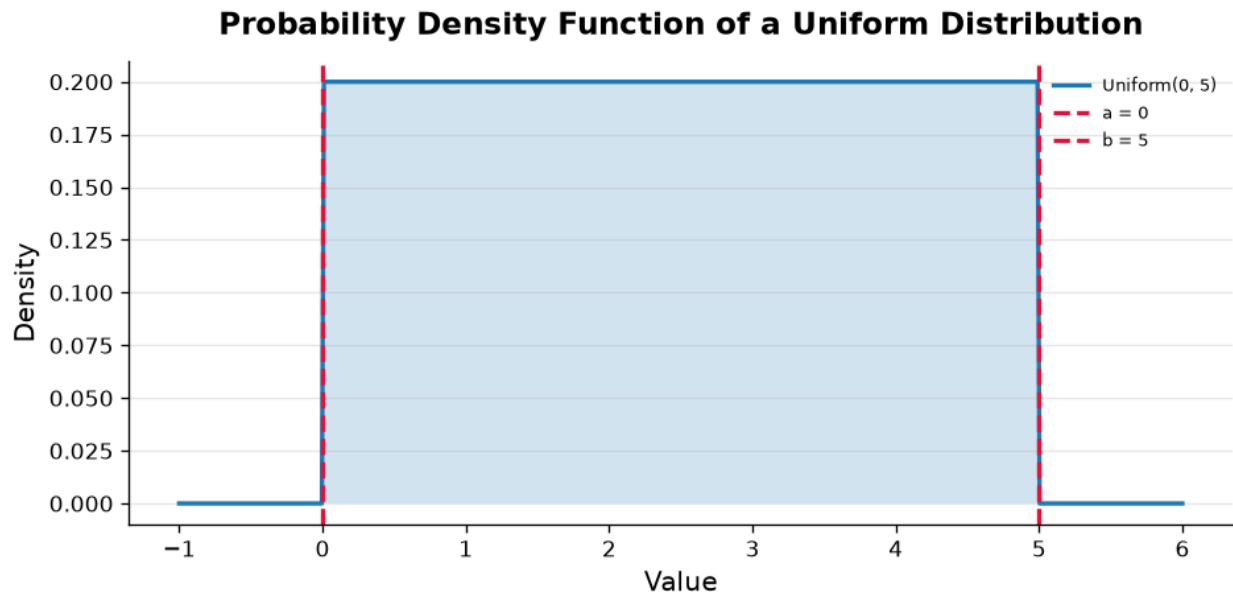
ax.set_ylabel(
    "Density",
    fontsize=12
)

# Style
ax.grid(axis="y", alpha=0.3)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

ax.legend(frameon=False, fontsize=8)

plt.tight_layout()
plt.show()

```



An integration exercise will show that the probability of an observation between c and d , where $a < c < d < b$, is equal to $\frac{d-c}{b-a}$, which is intuitively correct given that the essence of a uniform distribution is that every possible observation between the lower and upper bounds is equally likely. Let's test:

```
a = 0
b = 5
c = 2
d = 3
stats.uniform.cdf(d, loc=a, scale=b - a) - stats.uniform.cdf(c, loc=a,
scale=b - a)
np.float64(0.19999999999999996)
```